

Documento de Entrega — Tech Challenge FIAP

Sistema Integrado de Atendimento e Execução de Serviços para Oficina Mecânica

1. Identificação do Grupo

Nome do grupo: Grupo 97

Participante: Pedro Paulo dos Santos Silva

Username no Discord: pedropaulo6447

2. Repositório

Link do repositório privado: <https://github.com/ppdssdev/tech-challenge-oficina>

3. Documentação DDD

Link do Miro ou ferramenta equivalente:

https://miro.com/app/board/uXjVHAB8KJA=

Além da documentação visual, o repositório contém os seguintes documentos técnicos de apoio:

- [README.md](#)
- [docs/architecture.md](#)
- [docs/requirements-checklist.md](#)
- [docs/api-examples.http](#)

Também foram preparados como artefatos complementares da entrega:

- Documento de Event Storming;
- Linguagem Ubíqua;
- Relatório de Vulnerabilidades;
- Roteiro do Vídeo.

4. Resumo da Solução

O projeto implementa um MVP de backend para uma oficina mecânica de médio porte, com foco na gestão de clientes, veículos, serviços, peças, estoque e ordens de serviço.

A solução foi desenvolvida em **Java 21** com **Spring Boot 3.5.14**, utilizando **PostgreSQL**, **Flyway**, **Spring Data JPA**, **Spring Security com JWT**, **Swagger/OpenAPI**, **Docker**, **Docker Compose**, **JUnit 5**, **H2 para testes** e **JaCoCo** para verificação de cobertura.

O sistema permite registrar e acompanhar o ciclo completo de uma ordem de serviço, desde a entrada do veículo na oficina até a entrega final ao cliente. O fluxo principal contempla criação da OS, identificação do cliente, associação do veículo, inclusão de serviços e peças, cálculo automático do orçamento, aprovação, baixa de estoque, execução, finalização e entrega.

A arquitetura adotada foi um **monólito Spring Boot em camadas**, adequado ao escopo de MVP e ao prazo do Tech Challenge. A separação interna do código organiza responsabilidades em entrada REST, serviços de aplicação, domínio, infraestrutura de persistência, segurança e configuração.

O projeto aplica **DDD de forma pragmática**, concentrando as principais regras de negócio no domínio. O agregado central é **WorkOrder**, responsável por proteger o ciclo de vida da ordem de serviço e suas transições de status. Também foram modelados Value Objects para **DocumentNumber** e **VehiclePlate**, garantindo normalização e validação de CPF/CNPJ e placa diretamente no domínio.

5. Objetivos Atendidos

A solução atende aos principais objetivos do desafio:

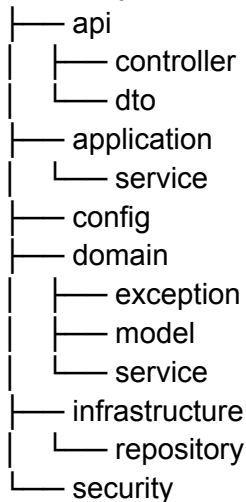
- controlar o fluxo de atendimento de uma oficina mecânica;
- registrar clientes e veículos;
- criar ordens de serviço;
- controlar serviços solicitados e peças utilizadas;
- calcular orçamento automaticamente;
- permitir aprovação do orçamento;
- controlar status da OS;
- reduzir estoque no momento adequado;
- permitir consulta pública do andamento da OS;
- proteger APIs administrativas com autenticação JWT;
- documentar a API com Swagger/OpenAPI;
- executar a aplicação com Docker e Docker Compose;
- garantir qualidade com testes automatizados;

- registrar análise de vulnerabilidades.

6. Arquitetura da Aplicação

A aplicação foi organizada como um monólito em camadas, com a seguinte estrutura principal de pacotes:

src/main/java/br/com/fiap/techchallenge/oficina



6.1 Responsabilidades por Camada

Camada	Responsabilidade
api	Controllers REST, DTOs de entrada/saída e tratamento global de erros.
application	Orquestração dos casos de uso, transações e integração entre repositórios e domínio.
domain	Entidades, Value Objects, regras de negócio, validações e ciclo de vida da OS.
infrastructure	Repositórios Spring Data JPA e consultas de persistência.
security	JWT, autenticação, filtro de segurança e integração com usuários administrativos.
config	Configurações de segurança, Swagger/OpenAPI e bootstrap do usuário admin.

Essa organização mantém o projeto simples para um MVP, mas com separação suficiente para evoluir o sistema sem misturar regras de negócio com detalhes de API, segurança ou banco de dados.

7. DDD Aplicado no Projeto

O DDD foi utilizado de maneira objetiva, priorizando a clareza do domínio da oficina e a proteção das regras mais importantes.

7.1 Agregado Principal

O agregado central do projeto é `WorkOrder`.

Ele representa a **Ordem de Serviço** e concentra regras como:

- criação da OS em estado `RECEIVED`;
- inclusão de serviços;
- inclusão de peças e insumos;
- cálculo automático dos totais;
- mudança para `WAITING_APPROVAL` quando existe orçamento;
- aprovação do orçamento;
- reserva/baixa de estoque das peças;
- início de execução;
- finalização;
- entrega do veículo;
- cálculo de tempo de execução.

Os itens `WorkOrderServiceItem` e `WorkOrderPartItem` pertencem ao agregado `WorkOrder` e compõem o orçamento da OS.

7.2 Value Objects

Foram implementados Value Objects para proteger regras de validação e normalização:

Value Object	Responsabilidade
<code>DocumentNumber</code>	Normalizar e validar CPF/CNPJ do cliente.
<code>VehiclePlate</code>	Normalizar e validar placas brasileiras nos formatos antigo e Mercosul.

Com isso, regras sensíveis não ficam espalhadas em controllers ou DTOs. Elas passam a fazer parte do próprio modelo de domínio.

7.3 Entidades Principais do Domínio

Entidade	Responsabilidade
----------	------------------

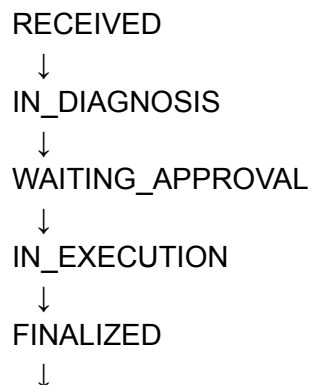
Customer	Representa o cliente da oficina, identificado por CPF/CNPJ.
Vehicle	Representa o veículo atendido, associado a um cliente.
ServiceCatalogItem	Representa um serviço oferecido pela oficina.
Part	Representa uma peça ou insumo, com preço, SKU, estoque e estoque mínimo.
WorkOrder	Representa a ordem de serviço e seu ciclo de vida.
AppUser	Representa o usuário administrativo utilizado na autenticação.

8. Ciclo de Vida da Ordem de Serviço

A OS possui os seguintes status:

Status	Significado
RECEIVED	Ordem de serviço recebida pela oficina.
IN_DIAGNOSIS	Veículo em diagnóstico.
WAITING_APPROVAL	Orçamento gerado e aguardando aprovação.
IN_EXECUTION	Orçamento aprovado e serviço em execução.
FINALIZED	Serviço finalizado pela oficina.
DELIVERED	Veículo entregue ao cliente.

Fluxo principal implementado:



DELIVERED

A transição para **WAITING_APPROVAL** ocorre automaticamente quando a OS possui orçamento calculado. A baixa de estoque ocorre somente na aprovação do orçamento, evitando reduzir estoque antes da autorização do cliente.

9. Funcionalidades Implementadas

9.1 Ordem de Serviço

- Criação de OS com cliente e veículo.
- Cadastro automático de cliente quando o CPF/CNPJ ainda não existe.
- Cadastro automático de veículo quando a placa ainda não existe.
- Inclusão de serviços solicitados.
- Inclusão de peças e insumos.
- Cálculo automático do orçamento.
- Alteração automática de status.
- Aprovação do orçamento.
- Baixa de estoque na aprovação.
- Finalização da OS.
- Entrega da OS.
- Consulta pública de status por código da OS e documento do cliente.

9.2 Gestão Administrativa

- CRUD de clientes.
- CRUD de veículos.
- CRUD de serviços.
- CRUD de peças e insumos.
- Ajuste manual de estoque.
- Consulta de peças abaixo do estoque mínimo.
- Listagem de ordens de serviço.
- Detalhamento de ordens de serviço.
- Filtro de OS por status.
- Métrica de tempo médio de execução.

9.3 Segurança

- Login administrativo com JWT.
- Proteção das rotas administrativas.
- Consulta pública restrita por código da OS e CPF/CNPJ.
- Senhas armazenadas com **BCryptPasswordEncoder**.
- Aplicação stateless com Spring Security.

- Endpoints públicos limitados a login, documentação, healthcheck e consulta pública da OS.
-

10. Principais Endpoints

10.1 Autenticação

POST /api/v1/auth/login

10.2 Clientes

POST /api/v1/admin/customers

GET /api/v1/admin/customers

GET /api/v1/admin/customers/{id}

PUT /api/v1/admin/customers/{id}

DELETE /api/v1/admin/customers/{id}

10.3 Veículos

POST /api/v1/admin/vehicles

GET /api/v1/admin/vehicles

GET /api/v1/admin/vehicles/{id}

PUT /api/v1/admin/vehicles/{id}

DELETE /api/v1/admin/vehicles/{id}

10.4 Serviços

POST /api/v1/admin/services

GET /api/v1/admin/services

GET /api/v1/admin/services/{id}

PUT /api/v1/admin/services/{id}

DELETE /api/v1/admin/services/{id}

10.5 Peças e Estoque

POST /api/v1/admin/parts

GET /api/v1/admin/parts

GET /api/v1/admin/parts/below-minimum-stock

GET /api/v1/admin/parts/{id}

PUT /api/v1/admin/parts/{id}

POST /api/v1/admin/parts/{id}/stock/increase

POST /api/v1/admin/parts/{id}/stock/decrease

DELETE /api/v1/admin/parts/{id}

10.6 Ordens de Serviço

POST /api/v1/admin/work-orders
GET /api/v1/admin/work-orders
GET /api/v1/admin/work-orders?status=WAITING_APPROVAL
GET /api/v1/admin/work-orders/{id}
POST /api/v1/admin/work-orders/{id}/diagnosis/start
POST /api/v1/admin/work-orders/{id}/diagnosis/notes
POST /api/v1/admin/work-orders/{id}/items
POST /api/v1/admin/work-orders/{id}/approve
POST /api/v1/admin/work-orders/{id}/finish
POST /api/v1/admin/work-orders/{id}/deliver

10.7 Consulta Pública

GET /api/v1/public/work-orders/{code}/status?document=...

10.8 Métricas

GET /api/v1/admin/metrics/average-execution-time

Com filtros opcionais:

GET
/api/v1/admin/metrics/average-execution-time?from=2026-06-01T00:00:00Z&to=2026-06-30T23:59:59Z

11. Controle de Estoque e Concorrência

O estoque foi tratado como ponto crítico do domínio.

A entidade **Part** concentra regras como:

- incremento de estoque;
- baixa de estoque;
- bloqueio de quantidade inválida;
- impedimento de estoque negativo;
- verificação de estoque mínimo;
- controle de preço e SKU.

Para lidar com concorrência, foram usados dois mecanismos:

Mecanismo

Uso

@Version em Part

Permite detecção de conflito otimista em atualizações concorrentes.

Lock pessimista em consultas críticas

Protege operações de baixa/incremento e aprovação de orçamento.

Na aprovação da OS, o sistema valida a disponibilidade total das peças pendentes antes de realizar qualquer baixa. Isso evita baixa parcial quando não há estoque suficiente para atender todo o orçamento.

12. Banco de Dados

O banco utilizado é o **PostgreSQL**.

A escolha foi adequada ao domínio porque o sistema exige:

- integridade referencial entre cliente, veículo, OS, serviços e peças;
- transações consistentes;
- controle seguro de estoque;
- campos monetários com precisão decimal;
- suporte a UUID;
- consultas administrativas e métricas.

O versionamento do schema é feito com **Flyway**, por meio das migrations:

V1__create_schema.sql

V2__seed_catalog.sql

V3__add_part_version.sql

O projeto também utiliza dados iniciais para serviços e peças, facilitando a execução e demonstração do fluxo principal da OS.

13. Qualidade e Testes

O projeto possui testes automatizados cobrindo domínio, aplicação, API, tratamento de exceções e fluxo integrado.

Foram identificados os seguintes tipos de teste:

Tipo

Exemplo

Testes de domínio	<code>WorkOrderDomainTest</code> , <code>PartInventoryTest</code> , <code>ValueObjectTest</code>
Testes de validação	<code>DocumentValidatorTest</code> , <code>VehiclePlateValidatorTest</code>
Testes de aplicação	<code>WorkOrderApplicationServiceTest</code>
Testes de API e fluxo	<code>WorkOrderFlowIntegrationTest</code>
Testes de erro HTTP	<code>ApiExceptionHandlerTest</code>

Resultado do último relatório de testes presente no projeto:

Testes executados: 36

Falhas: 0

Erros: 0

Ignorados: 0

O JaCoCo está configurado no `pom.xml` para exigir cobertura mínima de **80% nos pacotes críticos de domínio** durante o comando:

```
mvn clean verify
```

14. Tratamento de Erros

A aplicação possui tratamento global de exceções, convertendo falhas em respostas HTTP adequadas.

Principais status utilizados:

Status HTTP	Uso
<code>400 Bad Request</code>	Payload inválido, parâmetro inválido ou erro de conversão.
<code>401 Unauthorized</code>	Credenciais inválidas ou JWT ausente/inválido.
<code>403 Forbidden</code>	Usuário autenticado sem permissão.
<code>404 Not Found</code>	Recurso inexistente ou consulta pública incompatível com o documento informado.
<code>409 Conflict</code>	Conflito de unicidade ou integridade.

422 Unprocessable Entity Violação de regra de negócio.

Esse tratamento melhora a previsibilidade da API e facilita a integração com clientes externos.

15. Segurança da Aplicação

Foram implementadas as seguintes medidas de segurança:

- autenticação administrativa com JWT;
- proteção das rotas `/api/v1/admin/**`;
- liberação controlada de rotas públicas;
- senha administrativa criptografada com BCrypt;
- aplicação stateless;
- consulta pública exigindo código da OS e documento do cliente;
- validação de CPF/CNPJ no domínio;
- validação de placa no domínio;
- imagem Docker final executando com usuário não-root;
- atualização de pacotes Alpine no build da imagem;
- revisão de dependências após scan de vulnerabilidades.

As credenciais padrão abaixo são destinadas apenas ao ambiente local de desenvolvimento:

username: admin
password: admin123

Para qualquer ambiente fora do local, a recomendação é substituir essas credenciais por variáveis de ambiente seguras.

16. Relatório de Vulnerabilidades

Foi utilizado o **Trivy via Docker** para analisar o filesystem do projeto e a imagem Docker final.

Resumo do resultado final:

Alvo	Resultado
<code>pom.xml</code>	0 vulnerabilidades detectadas

Sistema operacional Alpine da imagem final	0 vulnerabilidades detectadas
JAR da aplicação	1 vulnerabilidade média residual
Vulnerabilidades críticas	0
Vulnerabilidades altas	0

A vulnerabilidade residual identificada está associada à biblioteca:

com.fasterxml.jackson.core:jackson-databind
CVE-2026-54515
Severidade: média
Versão instalada: 2.21.4
Versões corrigidas indicadas: 3.1.4, 2.18.9, 2.21.5

A decisão registrada foi manter a versão **2.21.4** no momento da validação, pois ela já removeu vulnerabilidades de maior severidade anteriormente reportadas. A recomendação é reexecutar o scan antes da entrega final e atualizar para uma versão corrigida assim que estiver disponível e compatível com o projeto.

17. Como Executar o Projeto

17.1 Pré-requisitos

- Docker;
- Docker Compose.

17.2 Subir o Ambiente Completo

Na raiz do projeto:

```
docker compose up --build
```

A API ficará disponível em:

<http://localhost:8080>

O PostgreSQL ficará disponível em:

localhost:5432

Credenciais locais do banco:

database: oficina_db

username: oficina

password: oficina

18. Swagger/OpenAPI

Com a aplicação em execução, acessar:

<http://localhost:8080/swagger-ui.html>

Fluxo recomendado:

1. Executar login em `POST /api/v1/auth/login`.
2. Copiar o `accessToken`.
3. Clicar em `Authorize`.
4. Informar o token no formato:

Bearer SEU_TOKEN_AQUI

19. Executar Testes

Para executar todos os testes:

```
mvn test
```

Para executar testes, empacotamento e verificação de cobertura:

```
mvn clean verify
```

Relatório JaCoCo:

<target/site/jacoco/index.html>

20. Executar Scan de Vulnerabilidades

Criar diretório para os relatórios:

```
mkdir -p target/security
```

Scan do filesystem:

```
docker run --rm -v "$PWD:/workspace" -v "$HOME/.cache/trivy:/root/.cache" \
  aquasec/trivy:latest fs --offline-scan --format table \
  --output /workspace/target/security/trivy-fs.txt /workspace
```

Build da imagem:

```
docker build --progress=plain -t oficina-api:local .
```

Scan da imagem:

```
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  -v "$HOME/.cache/trivy:/root/.cache" \
  -v "$PWD/target/security:/reports" \
  aquasec/trivy:latest image --offline-scan --format table \
  --output /reports/trivy-image.txt oficina-api:local
```

21. Requisitos Atendidos

Requisito	Implementação
Backend monolítico	Projeto único Spring Boot em camadas.
DDD	Domínio explícito com entidades, agregado, Value Objects e regras de negócio.
Criação de OS	<code>POST /api/v1/admin/work-orders</code> .
Identificação por CPF/CNPJ	<code>DocumentNumber</code> e <code>DocumentValidator</code> .
Cadastro de cliente	CRUD administrativo e criação automática na OS.
Cadastro de veículo	CRUD administrativo e criação automática na OS.
Inclusão de serviços	Lista de serviços na criação da OS e endpoint de adição de itens.
Inclusão de peças	Lista de peças na criação da OS e endpoint de adição de itens.

Orçamento automático	Cálculo em <code>WorkOrder</code> .
Aprovação de orçamento	<code>POST /api/v1/admin/work-orders/{id}/approve</code> .
Controle de status	<code>WorkOrderStatus</code> .
Baixa de estoque	Realizada na aprovação da OS.
Consulta pública	<code>GET /api/v1/public/work-orders/{code}/status?document=...</code>
CRUD clientes	<code>/api/v1/admin/customers</code> .
CRUD veículos	<code>/api/v1/admin/vehicles</code> .
CRUD serviços	<code>/api/v1/admin/services</code> .
CRUD peças	<code>/api/v1/admin/parts</code> .
Controle de estoque	Endpoints de incremento, baixa e peças abaixo do mínimo.
Métrica de execução	<code>/api/v1/admin/metrics/average-execution-time</code> .
JWT	Login e filtro JWT para rotas administrativas.
Swagger/OpenAPI	<code>/swagger-ui.html</code> .
Dockerfile	Imagem multi-stage com runtime Alpine.
Docker Compose	API + PostgreSQL.
Testes automatizados	Testes de domínio, aplicação, API e integração.
Cobertura mínima	JaCoCo com regra de 80% nos pacotes críticos de domínio.
Relatório de vulnerabilidades	Scan com Trivy documentado.

22. Limitações Assumidas do MVP

Por se tratar de um MVP acadêmico, algumas decisões foram simplificadas:

- não há envio real de notificação para o cliente;
- o orçamento é disponibilizado pela API e representado pelo status `WAITING_APPROVAL`;
- não há módulo financeiro completo;
- não há integração com gateway de pagamento;

- não há painel frontend;
- não há gestão avançada de perfis administrativos;
- as credenciais padrão são apenas para execução local;
- a consulta pública da OS utiliza código da OS e documento do cliente como mecanismo simples de restrição.

Essas limitações não comprometem o objetivo principal do desafio, que é demonstrar o fluxo de atendimento e execução de serviços com domínio estruturado, APIs protegidas, persistência, testes, documentação e análise de vulnerabilidades.

23. Checklist Final Antes do Envio

Antes da submissão final, validar:

- Nome do grupo preenchido.
 - Participantes preenchidos.
 - Usernames no Discord preenchidos.
 - Link do repositório privado preenchido.
 - Usuário `soat-architecture` adicionado ao repositório.
 - Link do Miro ou ferramenta equivalente preenchido.
 - Documento de Event Storming finalizado.
 - Linguagem Ubíqua finalizada.
 - Relatório de Vulnerabilidades finalizado.
 - Roteiro do Vídeo finalizado.
 - Último `mvn clean verify` executado com sucesso.
 - Docker build validado.
 - Aplicação testada via Swagger.
 - PDF de entrega gerado.
 - Vídeo gravado conforme orientação da FIAP.
 - Link do vídeo incluído na plataforma de entrega, se solicitado.
 - Repositório mantido privado até a correção, conforme orientação do desafio.
-

24. Conclusão

O projeto entrega um MVP funcional para gestão de uma oficina mecânica, cobrindo o ciclo principal da ordem de serviço, desde a entrada do veículo até sua entrega ao cliente.

A implementação utiliza um monólito Spring Boot em camadas, com DDD aplicado de forma pragmática nos pontos mais importantes do domínio. O agregado `WorkOrder` centraliza o ciclo da OS, enquanto Value Objects como `DocumentNumber` e `VehiclePlate` protegem validações essenciais. O controle de estoque foi tratado com atenção transacional e mecanismos de concorrência.

Além das funcionalidades de negócio, o projeto contempla autenticação JWT, documentação Swagger/OpenAPI, execução com Docker, banco PostgreSQL versionado com Flyway, testes automatizados, cobertura mínima de domínio crítico e relatório de vulnerabilidades com Trivy.

Dessa forma, a solução atende aos requisitos principais do Tech Challenge e apresenta uma base organizada, segura e evolutiva para um sistema administrativo de oficina mecânica.